Experiment No. 4

Title: Implementation of problem using Dynamic Programming Approach

Batch: B3 **Roll No: 16014225801** **Experiment No.:4**

Aim: Implementation of problem using Dynamic Programming Approach

Resources needed: Text Editor, C/C++ IDE

Theory:

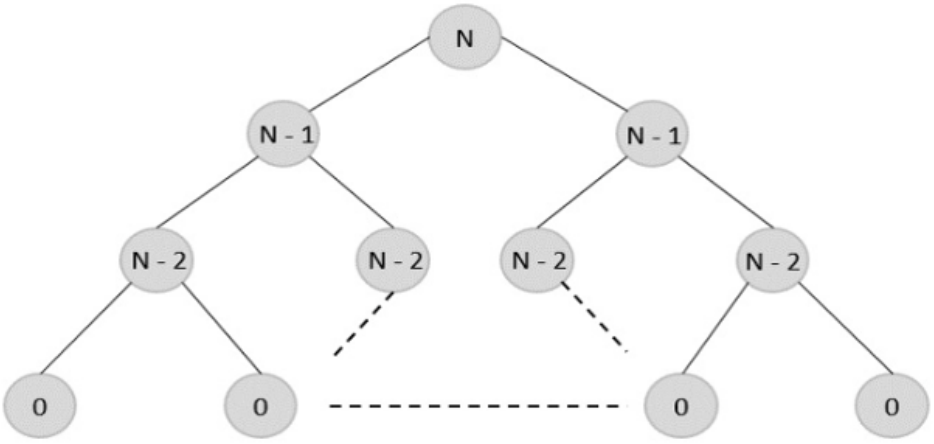
Dynamic Programming in C++ is a technique used to solve problems by breaking them down into smaller subproblems and then combining the solutions of these subproblems to form the complete solution. The approach is based on the following properties:

Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems. But unlike divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.

Mostly, dynamic programming algorithms are used for solving optimization problems. Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems. The solutions of sub-problems are combined in order to achieve the best optimal final solution. This paradigm is thus said to be using Bottom-up approach.

So we can conclude that −

- The problem should be able to be divided into smaller overlapping sub-problem.
- Final optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- Dynamic algorithms use memorization.



Overlapping Sub-Problems

Similar to Divide-and-Conquer approach, Dynamic Programming also combines solutions to sub-problems. It is mainly used where the solution of one sub-problem is needed repeatedly. The computed solutions are stored in a table, so that these dont have to be re-computed. Hence, this technique is needed where overlapping sub-problem exists.

For example, Binary Search does not have overlapping sub-problem. Whereas recursive program of Fibonacci numbers have many overlapping sub-problems.

Optimal Sub-Structure

A given problem has Optimal Substructure Property, if the optimal solution of the given problem can be obtained using optimal solutions of its sub-problems.

For example, the Shortest Path problem has the following optimal substructure property −

If a node x lies in the shortest path from a source node u to destination node v, then the shortest path from u to v is the combination of the shortest path from u to x, and the shortest path from x to v.

The standard All Pair Shortest Path algorithms like Floyd-Warshall and Bellman-Ford are typical examples of Dynamic Programming.

Steps of Dynamic Programming Approach

- Dynamic Programming algorithm is designed using the following four steps −
- Characterize the structure of an optimal solution.
- Recursively define the value of an optimal solution.
- Compute the value of an optimal solution, typically in a bottom-up fashion.
- Construct an optimal solution from the computed information.

Dynamic Programming is an algorithmic technique with the following properties.

- It is mainly an optimization over plain recursion. Wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using Dynamic Programming.
- The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. This simple optimization typically reduces time complexities from exponential to polynomial.

Activity:

Problem Statement:

Pankaj is a very intelligent student studying in one of the best colleges of this country. He's good enough to challenge the smartest minds in the country, but even the smart ones make mistakes and so did Pankaj - he fell in love, ah. He was so deeply in love that he decided to propose the love of his life. What he didn't know was the fact that there is a different species called the "*in-laws*" and against them no amount of intelligence, smartness or anything works!

But, this was no ordinary person - this was Pankaj - he decided to take the challenge of his in-laws. So, in order to verify the compatibility of the two love birds, they gave them a sequence to solve - given a sequence, they had to tell the longest increasing subsequence of the given sequence.

But, this was a tricky game - since, any sequence could have multiple increasing subsequences of the same length - so, the answer they were giving were different almost all the time, which got the young couple worried.
So, in order increase their compatibility Pankaj proposes that instead of telling them the sequence, this time the couple would tell them the of the longest increasing subsequence and not the sequence.
To give further twist by the in-laws, they asked Pankaj to answer the length **not** in decimal numbers, but in binary numbers!

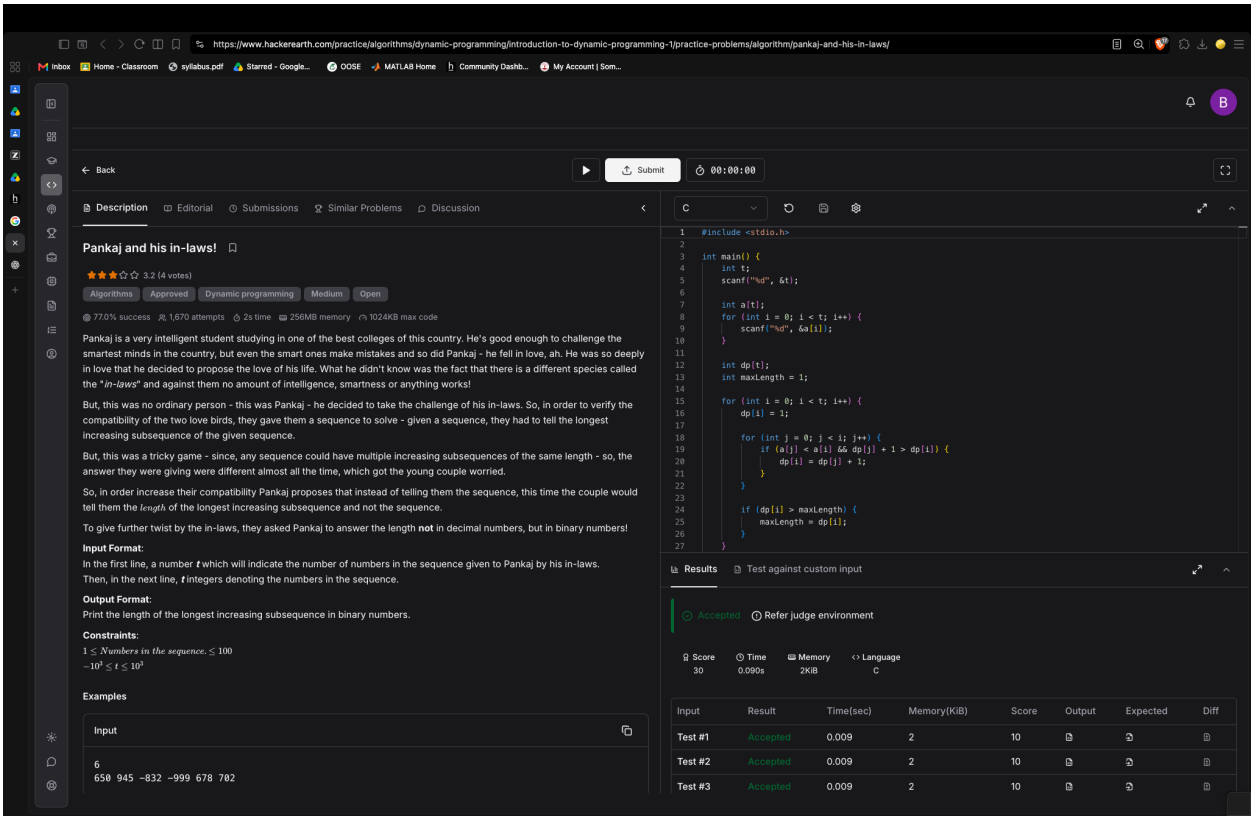
Input Format:
In the first line, a number ***t*** which will indicate the number of numbers in the sequence given to Pankaj by his in-laws. Then, in the next line, ***t*** integers denoting the numbers in the sequence.

Output Format:
Print the length of the longest increasing subsequence in binary numbers.

Constraints:
$1 \leq Numbers\ in\ the\ sequence. \leq 100$
$-10^3 \leq t \leq 10^3$

Sample Input	Sample Output
6 650 945 -832 -999 678 702	11

Solution:



Use Case Implementation

The given program can be used in applications where a sequence of values needs to be analyzed to find the longest continuously increasing trend while maintaining the original order of elements. For example, it can be used to analyze stock prices, student marks, temperature readings, sales figures, or other time-series data. In this experiment, the program finds the length of the Longest Increasing Subsequence using Dynamic Programming and then converts the resulting length from decimal to binary. The dp[i] array stores the length of the longest increasing subsequence ending at the ith element. For every element, previously calculated values are checked, avoiding repeated computation of overlapping subproblems. Thus, the program demonstrates both optimal substructure and overlapping subproblems, which are fundamental properties of Dynamic Programming.

Solution

The program first accepts the number of elements and stores the sequence in an array. A dp array is initialized, where dp[i] represents the length of the longest increasing subsequence ending at position i. Initially, every element forms a subsequence of length 1. For every element a[i], the program checks all previous elements a[j]. If a[j] < a[i], the current subsequence can be extended, so dp[i] is updated using dp[j] + 1. The maximum value among all dp[i] gives the

length of the Longest Increasing Subsequence. Finally, this decimal length is converted into binary using repeated division by 2 and printed in reverse order.

Output:

```
Input Sequence: 10 22 9 33 21 50 41 60
Length of Longest Increasing Subsequence: 5
LIS Length in Binary: 101
```

Conclusion:

The experiment successfully implemented the Longest Increasing Subsequence using Dynamic Programming. The approach efficiently handled overlapping subproblems and optimal substructure to determine the LIS length. The obtained length was then converted into binary form, demonstrating the practical application of Dynamic Programming in sequence optimization problems.

References:

1. Steven S. Skiena Miguel A. Revilla,"Programming challenges, The Programming Contest Training Manual", Springer, 2006
2. T.H. Coreman ,C.E. Leiserson,R.L. Rivest, and C. Stein, " Introduction to algorithms", 3rd Edition 2009, Prentice Hall India Publication
3. Antti Laaksonen, "Guide to Competitive Programming",Springer,2018
4. https://www.hackerearth.com/practice/algorithms/dynamic-programming/introduction-to-dynamic-programming-1/practice-problems/algorithm/pankaj-and-his-in-laws/
5. https://drive.google.com/drive/folders/1c9mCFdPtIEW3TAs0tWpC9x2xJQwQ1E3j?usp=drive_link